

Learningstory Requirements & Wireframe

Inleiding

In dit project werk ik aan het analyseren van de huidige situatie bij Reemo en het ontwerpen van een voorstel voor een nieuwe applicatie en een dashboard. Reemo is een IT-dienstverlener die organisaties helpt bij het bouwen en opschalen van software. Hoewel het bedrijf inhoudelijk sterk is en inmiddels een team van ongeveer 25 medewerkers heeft, zijn de interne administratieve processen niet meegegroeid met het succes van de organisatie.

Binnen Reemo worden cruciale processen, zoals de urenregistratie van consultants en de facturatie aan klanten, momenteel nog grotendeels handmatig uitgevoerd. Informatie staat verspreid over verschillende systemen, zoals losse Word-templates, PDF-bestanden, e-mails en zelfs WhatsApp-berichten. Dit zorgt ervoor dat stappen niet goed op elkaar aansluiten en er weinig overzicht is voor de gebruikers. Vaak wordt alleen het eindprobleem zichtbaar—zoals medewerkers die informatie niet kunnen vinden of facturatie die vertraagt—maar de werkelijke oorzaak ligt dieper: onduidelijke requirements en een gebrek aan inzicht in wat de gebruikers écht nodig hebben.

Mijn Persoonlijke Leerdoel

Als student wil ik met deze learning story leren hoe ik deze vage problemen vertaal naar een concreet softwarevoorstel. Mijn focus ligt hierbij op twee cruciale onderdelen van de analysefase:

1. **Requirements opstellen:** Leren hoe ik duidelijk beschrijf wat de applicatie en het dashboard moeten kunnen om de problemen bij Reemo op te lossen.
2. **Wireframes maken:** Leren hoe ik deze ideeën visueel uitwerk, zodat voor stakeholders zoals Maarten (CEO) en de developers direct zichtbaar wordt hoe de software eruit gaat zien en hoe zij ermee gaan werken.

Leerstrategie

Om dit leerdoel te bereiken, volg ik een gestructureerde aanpak. Ik begin met het lezen van literatuur over requirements-engineering en wireframing om een theoretische basis te leggen. Vervolgens onderzoek ik de huidige software en processen bij Reemo door middel van interviews en observaties, zodat ik de werkelijke behoeften boven tafel krijg. Op basis daarvan stel ik requirements op, die ik uiteindelijk vertaal naar een visueel ontwerp dat past bij de wensen van de gebruikers.

De theorie: requirements en de MoSCoW-methode

Om mijn leerdoel te behalen, ben ik eerst gaan kijken naar de theorie achter requirements en hoe je deze opstelt. Ik wilde begrijpen hoe je van een onduidelijke en rommelige situatie naar een duidelijk en gestructureerd plan gaat.

Requirements zijn volgens Visure Solutions (2024) de eisen en wensen waar een systeem aan moet voldoen. Je kunt dit zien als een soort bouwtekening van de software. Het beschrijft wat het systeem moet kunnen en hoe het moet werken. Zonder duidelijke requirements bestaat het risico dat je iets ontwikkelt dat niet aansluit op de behoefte van de gebruiker. Daarom is het belangrijk om deze zo concreet mogelijk te maken. In plaats van een vage wens zoals “meer overzicht”, moet je dit vertalen naar iets specifiek, zoals een dashboard waarin per klant alle relevante informatie zichtbaar is.

Het opstellen van goede requirements is belangrijk omdat het helpt om de doelen van een organisatie te koppelen aan de technische oplossing. Toolshero (2024) legt uit dat een business requirements analyse ervoor zorgt dat je niet alleen naar techniek kijkt, maar vooral naar wat het bedrijf nodig heeft. In het geval van Reemo is dat duidelijk zichtbaar. De eigenaar heeft veel kennis in zijn hoofd en werkt met losse systemen. Door requirements op te stellen, wordt deze kennis omgezet in een gestructureerde oplossing binnen de software.

Tijdens het opstellen van de requirements merkte ik dat er veel wensen tegelijk naar voren kwamen. Omdat je niet alles in één keer kunt bouwen, heb ik de MoSCoW-methode gebruikt om prioriteiten te stellen. Deze methode helpt om onderscheid te maken tussen wat echt noodzakelijk is en wat eventueel later kan worden toegevoegd (Toolshero, 2024).

Binnen deze methode betekent een ‘must have’ dat het systeem zonder deze functie niet werkt. Voor Reemo gaat het bijvoorbeeld om een centrale plek waar contracten en klantinformatie worden opgeslagen. ‘Should have’ zijn functies die belangrijk zijn, maar niet direct nodig voor de eerste versie, zoals automatische herinneringen voor urenregistratie. ‘Could have’ zijn extra functies die het systeem verbeteren, zoals visuele toevoegingen of kleine gebruiksgemakken. Tot slot zijn er ook onderdelen die bewust buiten de eerste versie worden gehouden. Dit zorgt ervoor dat de focus blijft liggen op het oplossen van het kernprobleem en dat het project overzichtelijk blijft.

De praktijk: interviews, observaties en feedback bij Reemo

Na de eerste analyse heb ik niet eerst uitgebreide wireframes gemaakt, maar ben ik direct begonnen met een prototype. Ik merkte dat de situatie bij Reemo nog vrij onduidelijk was en dat een werkend ontwerp sneller zou helpen om te zien wat wel en niet werkt.

Dit prototype gaf meteen een duidelijk beeld van hoe een centrale oplossing eruit zou kunnen zien. Tijdens het bouwen merkte ik al dat het probleem vooral zit in het gebrek aan overzicht en dat veel handmatige stappen eigenlijk geautomatiseerd kunnen worden.

Vervolgens heb ik het prototype laten zien aan Alexander, Esmee en Maarten om gerichte feedback op te halen. Deze feedback was belangrijk, omdat het liet zien wat er nog miste en waar de echte prioriteiten liggen.

Uit de feedback van de developers kwam naar voren dat het ontwerp te veel focus had op visuele elementen en te weinig op data. Ze gaven aan dat functies zoals inzicht in capaciteit en structuur belangrijker zijn dan animaties. Vanuit de business kant kwam juist de behoefte aan overzicht en gebruiksgemak. Zo gaf Esmee aan dat ze een centrale plek wil om developers te zoeken op basis van skills. Maarten gaf aan dat hij contracten makkelijker wil kunnen beheren, bijvoorbeeld via een drag-and-drop functie voor PDF-bestanden.

Door deze feedback werd duidelijk dat het prototype niet alleen een eerste ontwerp was, maar ook een middel om requirements scherper te maken. Het hielp om van algemene ideeën naar concrete functies te gaan die echt aansluiten op de praktijk van Reemo.

Op basis hiervan heb ik mijn requirements aangepast en opnieuw geprioriteerd.

Conclusie

Tijdens deze sprint, waarin ik mij heb gericht op het onderzoeken en opstellen van requirements, heb ik geleerd dat je als ontwerper niet alleen bezig bent met hoe iets eruitziet, maar vooral met het vertalen van een probleem naar een oplossing die echt werkt in de praktijk. Ik merkte dat je eigenlijk een brug moet slaan tussen wat de opdrachtgever wil en wat technisch haalbaar is.

In het begin had ik de neiging om direct te denken in oplossingen, zoals hoe het dashboard eruit moest zien. Tijdens het onderzoeken van de requirements werd duidelijk dat dit niet de juiste start is. Goede software begint niet bij een mooi ontwerp, maar bij het begrijpen van het probleem. In het geval van Reemo zit dat probleem vooral in het gebrek aan overzicht, de handmatige stappen en het feit dat informatie verspreid staat.

De MoSCoW-methode heeft mij geholpen om structuur aan te brengen in alle wensen die naar voren kwamen. In plaats van alles tegelijk te willen oplossen, kon ik onderscheid maken tussen wat echt nodig is en wat later kan worden toegevoegd. Hierdoor werd het ontwerp overzichtelijker en beter onderbouwd.

Wat ik vooral heb geleerd, is dat ontwerpen een proces is waarin je blijft bijsturen. Tijdens het werken aan het prototype en het ophalen van feedback merkte ik dat mijn eerste ideeën nog niet scherp genoeg waren. Pas door te testen en feedback te verwerken, werd duidelijk wat echt belangrijk is voor de gebruiker.

Onderstaande MoSCoW-analyse laat zien hoe ik de wensen en eisen heb gestructureerd en vertaald naar concrete onderdelen van het systeem.

Bronnen (APA)

Lean Six Sigma Partners. (z.d.). Muda, Mura en Muri: drie bronnen van verspilling. Geraadpleegd op 19 februari 2026, van <https://www.leansixsigmapartners.nl/blog/muda-mura-muri-3-bronnen-van-verspilling>.

Reaco. (2024, 15 januari). 10 tips om requirements van complexe bedrijfsprocessen uit te vragen. Geraadpleegd op 25 maart 2026, van <https://www.reaco.nl/blog/10-tips-om-requirements-van-complexe-bedrijfsprocessen-uit-te-vragen/>.

The Lean Six Sigma Company. (z.d.). Poka-Yoke. Geraadpleegd op 19 februari 2026, van <https://www.theleansixsigmacompany.com/nl/bibliotheek/poka-yoke/>.

Toolshero. (2024, 12 februari). Business Requirements Analyse en de MoSCoW methode. Geraadpleegd op 25 maart 2026, van <https://www.toolshero.nl/project-management/business-requirements-analyse/>.

Visure Solutions. (2024). Gids voor Behoeftanalyse en Requirements Management. Geraadpleegd op 25 maart 2026, van <https://visuresolutions.com/nl/alm-gids/behoeftanalyse/>.

Reaco. (2024). 10 tips om requirements van complexe bedrijfsprocessen uit te vragen. <https://www.reaco.nl/blog/10-tips-om-requirements-van-complexe-bedrijfsprocessen-uit-te-vragen/>

Toolshero. (2024). Business Requirements Analyse en de MoSCoW methode. <https://www.toolshero.nl/project-management/business-requirements-analyse/>

Visure Solutions. (2024). Gids voor Behoeftanalyse en ALM. <https://visuresolutions.com/nl/alm-gids/behoeftanalyse/>

Link naar miro board:

https://miro.com/app/board/uXjVGs8Kf5M=?share_link_id=582009025495

Bijlage

Must Have

Centraal dashboard (per klant overzicht)

Berekening van verwachte omzet (uren $\times$ tarief)

Drag & drop upload voor contracten (PDF)

Koppeling tussen uren, contracten en tarieven

Should Have

CV-database met zoekfunctie (skills / naam)

Inzicht in capaciteit van freelancers

Validatie op uren (bijv. max uren per dag/week)

Automatische reminders voor uren

Could Have

Could Have

Klantlogo's in dashboard

Kleurcodes voor developers (bijv. rollen/status)

CV automatisch omzetten naar standaard format



Basis koppeling met externe tools (later uitbreiden)

Won't Have

Won't Have

Volledige backend integraties (complexe systemen)

Geavanceerde animaties / transitions

Marketing / CRM systeem